

AXI4-Stream Protocol — A Study Guide (with the HDC stream wrapper as the worked example)

Companion to `AXI4_Lite_Protocol_Study.pdf`. That guide covered the memory-mapped control path; this one covers **AXI4-Stream**, the protocol used for continuous, high-throughput data flow — and shows exactly how `rtl/hdc_stream_wrapper.sv` implements it.

1. Why a second protocol?

AXI4-Lite is perfect for *control*: write a few registers, start a job, poll a status bit. But it pays a heavy price per word — every 32-bit transfer costs a full address + data + response handshake sequence, with the CPU executing load/store instructions for each one. Streaming an EMG window through the LEVELS registers takes several bus transactions *per window*, and the CPU is busy the whole time.

AXI4-Stream removes everything that isn't data:

	AXI4-Lite	AXI4-Stream
Addresses	yes (AWADDR/ARADDR)	none — data flows point-to-point
Direction	read <i>and</i> write	one-way per interface
Response channel	yes (BRESP/RRESP)	none
Transfer unit	one word per transaction	continuous beats, back-to-back
Typical master	CPU	DMA engine , ADC, another PL block

On Zynq, the usual arrangement is: the PS programs an **AXI-DMA** engine (itself configured over AXI4-Lite) with a memory buffer; the DMA then streams the buffer into the PL as AXI4-Stream with zero CPU involvement per beat. That is exactly the role `hdc_stream_wrapper` is built for.

2. The signals — it's the same handshake

AXI4-Stream has **one channel** per direction. The core signals:

Signal	Driven by	Meaning
<code>TDATA</code>	master	the payload (here 32 bits)
<code>TVALID</code>	master	"TDATA is valid now"
<code>TREADY</code>	slave	"I can accept a beat now"
<code>TLAST</code>	master	marks the final beat of a <i>frame</i> (packet)

A beat transfers on the rising clock edge where `TVALID && TREADY` . This is the identical VALID/READY rule from the AXI4-Lite guide — same obligations: once `TVALID` is up, the master holds data stable until the beat is accepted; the slave back-pressures by holding `TREADY` low.



`TLAST` adds framing: a stream is an endless sequence of beats, so `TLAST = 1` on a beat says "this beat completes one logical unit" — one network packet, one image line, or in our case **one EMG window**. (The full spec also defines `TKEEP` / `TSTRB` / `TID` / `TDEST` / `TUSER` for byte qualification and routing; simple point-to-point designs like ours omit them, which is spec-compliant.)

3. Framing in the HDC wrapper

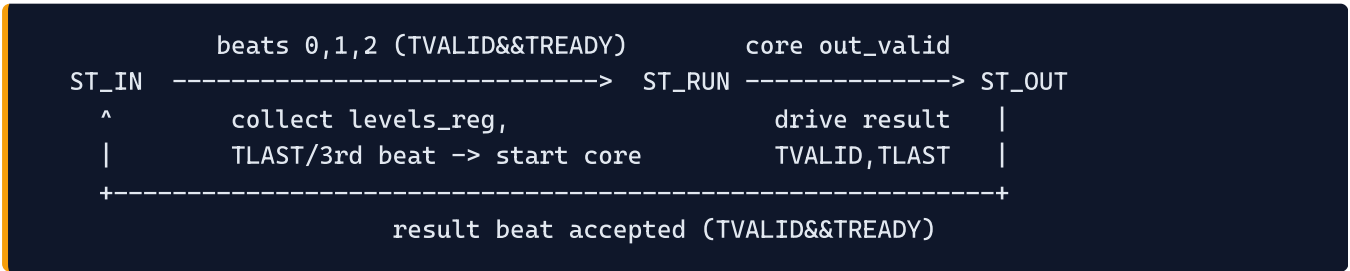
A window is the packed level grid: 20 levels × 4 bits = **80 bits**, which does not fit one 32-bit beat. So one window = **3 beats**, little-endian:

Beat	TDATA carries	TLAST
0	<code>levels_flat[31:0]</code>	0
1	<code>levels_flat[63:32]</code>	0
2	<code>levels_flat[79:64]</code> (upper 16 bits ignored)	1

The result stream is one beat per window — `(class_idx << 16) | class_dist` , `TLAST = 1` — the same encoding the AXI-Lite RESULT register uses, so software parses both paths identically.

4. How the wrapper implements it

`hdc_stream_wrapper.sv` is a three-state FSM around the verified `hdc_core_top` :



The protocol-relevant lines:

Input ready only while collecting. Back-pressure toward the DMA is just a state decode — while the core is busy or a result is waiting, the wrapper simply isn't ready and the master must hold its beat:

```
assign s_axis_tready = (st == ST_IN);
```

Accepting a beat and closing the frame. Each accepted beat is placed at its word offset; `TLAST` (or the beat counter, as a safety net) commits the window and starts the core:

```
if (s_axis_tvalid && s_axis_tready) begin
    ... levels_reg[beat*32 +: 32] <= s_axis_tdata ...
    if (s_axis_tlast || (beat == IN_BEATS-1)) begin
        core_start <= 1'b1;    // window complete -> run inference
        st         <= ST_RUN;
    end
end
end
```

Producing the result with correct back-pressure. `TVALID` is asserted with the data and **held** until the consumer takes it — the rule from Section 2:

```
ST_RUN: if (core_out_valid) begin
    m_axis_tdata <= {class_idx, class_dist};    // packed
    m_axis_tvalid <= 1'b1; m_axis_tlast <= 1'b1;
    st <= ST_OUT;
end
ST_OUT: if (m_axis_tvalid && m_axis_tready) begin
    m_axis_tvalid <= 1'b0; m_axis_tlast <= 1'b0;
    st <= ST_IN;
end
end
```

5. How the testbench torture-tests the handshake

A handshake bug (sampling data when only `TVALID` is high, dropping `TVALID` early, ignoring `TREADY`) hides when both sides are always ready. So `tb/tb_stream_cosim.sv` deliberately breaks the easy timing:

- **Master-side gaps** — `send_window` inserts a random 0–3 idle cycles before every beat, so the slave sees `TVALID` arrive at unpredictable times.
- **Slave-side back-pressure** — `m_axis_tready` is randomized every cycle (~60 % duty), so the wrapper must *hold* its result beat, sometimes for several cycles.
- Every beat is only counted on a true `TVALID && TREADY` edge, and every result is compared against the Python golden (`class_idx` **and** `class_dist` , plus `TLAST`).

200/200 streamed inferences match the golden under this randomized timing — which verifies the datapath *and* the protocol discipline together.

6. Checklist / glossary

- **Beat** = one TVALID && TREADY clock edge; the only moment data moves.
- **Frame / packet** = beats up to and including a TLAST = 1 beat.
- **Master never waits for TREADY to assert TVALID**; slave may assert TREADY whenever.
- **Hold the beat**: once TVALID is up, data/ TLAST stay stable until accepted.
- **Back-pressure** propagates upstream automatically: slave not ready → master stalls → (in a real system) the DMA pauses reading memory.
- **No addresses, no responses** — routing and error handling, if needed, come from TDEST / TUSER or the surrounding system, not the stream itself.
- On Zynq: **AXI-DMA** converts memory buffers ↔ AXI4-Stream; the PS only programs descriptors, the per-beat work is free.